

BeabaProgramAcao

Prof. Edson de Oliveira



@BeabaProgramAcao



@BeabaDaProgramacaoEds

E-books

Módulo 1 – Lógica de Programação Aplicada em Python e Linguagem C

<https://go.hotmart.com/A106714206Q>

Slicing – Fatiamiento de Listas e Strings | Métodos de Strings

<https://go.hotmart.com/Q106816089F>

Subalgoritmos em Python: Funções, procedimentos, parâmetros e Módulos

<https://go.hotmart.com/Q106816089F>

Todos com exercícios propostos e corrigidos



Dicionários

- Métodos mais relevantes
- Tabelas:
 - Dicionários de dicionários
 - Dicionários com listas
 - Lista de dicionários

Definições

Introdução

Dicionários são **estruturas de dados** em Python que armazenam **pares de informações**:

chave:valor (key:value)

Um **dicionário** é uma coleção **desordenada e mutável** de dados. Ele armazena pares de dados por **chave: valor** e é delimitado entre chaves.

Criando um dicionário vazio:

Há duas formas de criar um Dicionário vazio:

Forma 1:

```
nome_dicionario = dict()
```

ou

Forma 2:

```
nome_dicionario = {}
```

Sintaxe:

Para criarmos um dicionário com keys e values (campos e valores) valores iniciais, usamos a seguinte sintaxe:

Sintaxe:

```
nome_dicionario = {  
    "key1": value1,  
    "key2": value2,  
    "keyN": valueN,  
}
```

Sinônimos:

keys → campos → chaves

values → valores → conteúdos

Aspas simples (') ou duplas (") tem o mesmo efeito

Criando um dicionário:

Exemplo:

```
Nome do dicionário → aluno = {  
    "nome": "Carlos",  
    "idade": 21, ← value  
    "curso": "DS"  
} ← items
```

key

Observações:

- Cada par de chaves { } cria um dicionário.
- Os *values* podem ser de qualquer tipo.
- Dentro do par de chaves estão todos os *items*.
- O dois-pontos (:) separa a *key* do *value*.

Acessando valores no dicionário:

Use o formato: `dicionario["key"]`

Exemplos:

```
if aluno["nome"] == "Carlos": print("Ok!") # Se Carlos exibe Ok!
aluno["idade"] = int(input("Idade:"))      # Digita na key idade
print(aluno.get("curso"))                  # Exibe: DS
print(aluno.get("nota"))                   # None - Não há esta key
print(aluno["nota"])                       # ERRO: (KeyError)
```

A função `get()` é utilizada para recuperar o valor associado a uma determinada key.

Se a key **não existir** no dicionário, ao invés de gerar um erro (**KeyError**), ela retorna um valor padrão: **None** e não aborta a execução do programa

Modificando valores no dicionário:

Para alterar um valor, referencie a key entre colchetes e aspas e atribua um novo valor.

Code:

```
aluno["idade"] = 30  
print(aluno)
```

Output:

```
{'nome': 'João', 'idade': 30, 'curso': 'DS'}
```

O valor também pode ser editado pelo usuário:

Code:

```
aluno["idade"] = int(input("Idade: "))  
print(aluno)
```

Resultado:

```
{'nome': 'João', 'idade': <valor digitado>, 'curso': 'DS'}
```

Adicionando novas Keys

Basta atribuir uma nova key com um valor.

Code:

```
aluno["nota"] = 8.7
```

Se a key **existir**, o value será **atualizado**, senão ela será **criada**.

Resultado:

```
{'nome': 'João', 'idade': 40, 'curso': 'DS', 'nota': 8.7}
```

Removendo itens

Há duas formas:

Formas:

```
aluno.pop("curso")      # Remove a chave 'curso'
del aluno["idade"]      # Remove a chave 'idade'
```

O **pop** exclui somente um item por vez
Caso a Key não exista no dicionário ocorrerá o erro `KeyError`.

O **del** pode inclusive excluir (desalocar da memória) a variável:

Code:

```
del aluno
```

A variável 'aluno' deixa de existir.

Métodos de dicionários

Alguns métodos aplicáveis em dicionário:

Método	Descrição	Exemplo
get()	Retorna o valor de uma chave	d.get("nome")
keys()	Retorna todas as chaves	d.keys()
values()	Retorna todos os valores	d.values()
items()	Retorna pares (chave, valor)	d.items()
update()	Atualiza o dicionário com outro	d.update({"nome": "Ana"})
pop()	Remove a chave especificada	d.pop("curso")
clear()	Remove todos os itens	d.clear()

Exemplos de métodos de dicionários

Seguem alguns exemplos de aplicação de métodos em dicionários:

Code:

```
aluno = {
    "nome": "Carlos",
    "idade": 21,
    "curso": "Engenharia de Software"
}

# .keys()
print(aluno.keys())

# .values()
print(aluno.values())

# .items()
for chave, valor in aluno.items():
    print(chave, ":", valor)

# .update()
aluno.update({"nome": "Ana", "idade": 22})

# .pop()
aluno.pop("curso")

# .clear()
aluno.clear()
```

Output:

```
dict_keys(['nome', 'idade', 'curso'])
```

```
dict_values(['Carlos', 21, 'Engenharia de Software'])
```

```
nome : Carlos
```

```
idade : 21
```

```
curso : Engenharia de Software
```

```
{'nome': 'Ana', 'idade': 22, 'curso': 'Engenharia de Software'}
```

```
{'nome': 'Ana', 'idade': 22}
```

```
{}
```

Dicionários de dicionários

Um dicionário pode ser encadeado em outro.
Neste exemplo é exibida a nota da Maria:

Code:

```
turma = {  
    "aluno1": {"nome": "João", "nota": 7.5},  
    "aluno2": {"nome": "Maria", "nota": 9.2},  
}  
  
# Exibe a Nota da Maria  
print("Nota da Maria: ", turma["aluno2"]["nota"])
```

Output:

Nota da Maria: 9.2

✓ Vantagens

- É simples chegar a cada informação

✗ Desvantagens

- Há a necessidade de saber os nomes das Keys de toda estrutura

Dicionários de dicionários

Exibindo os registros citando as keys:

Code:

```
turma = {
    "aluno1": {"nome": "João", "nota": 7.5},
    "aluno2": {"nome": "Maria", "nota": 9.2},
}

# Exibe os registros dos alunos citando as keys
print("\nREGISTROS CITANDO AS KEYS:")
for chave, dados in turma.items():
    print(f"ID: {chave}")
    print(f"Nome: {dados['nome']}")
    print(f"Nota: {dados['nota']}")
    print("-" * 20)
```

Output:

```
REGISTROS CITANDO AS KEYS:
ID: aluno1
Nome: João
Nota: 7.5
-----
ID: aluno2
Nome: Maria
Nota: 9.2
-----
```

✓ Vantagens

- Mais fácil de entender.
- Código mais curto e limpo.
- Ótimo quando a estrutura de dados é fixa.

✗ Desvantagens

- Pouco flexível: se mudar a estrutura (adicionar ou remover keys), precisa alterar o código.

Dicionários de dicionários

Exibindo os registros percorrendo as Keys

Code:

```
turma = {  
    "aluno1": {"nome": "João", "nota": 7.5},  
    "aluno2": {"nome": "Maria", "nota": 9.2},  
}  
  
# Exibe os registros dos alunos percorrendo as keys  
print("\nREGISTROS PERCORRENDO AS KEYS:")  
for id_aluno, dados in turma.items():  
    print(f"ID: {id_aluno}")  
    for campo, valor in dados.items():  
        print(f"{campo.capitalize()}: {valor}")  
    print("-" * 20)
```

Output:

```
REGISTROS PERCORRENDO AS KEYS:  
ID: aluno1  
Nome: João  
Nota: 7.5  
-----  
ID: aluno2  
Nome: Maria  
Nota: 9.2  
-----
```

✓ Vantagens

- **Extremamente flexível:** se adicionar mais keys no dicionário, ele já exibe tudo.
- Facilita trabalhar com **dados dinâmicos** ou lidos de arquivos/BD.
- Evita ter que reescrever o código para novos campos.

✗ Desvantagens

- Mais difícil interpretar.

Dicionários de dicionários (com subalgoritmo)

Code:

```
# --- Funções e Procedimentos ---
def exibir_registro_aluno(id_aluno: str, dados: dict) -> None:
    # Exibe os dados de um único aluno
    print(f"ID: {id_aluno}")
    for campo, valor in dados.items():
        print(f"{campo.capitalize()}: {valor}")
    print("-" * 20)

def exibir_turma(turma: dict) -> None:
    # Percorre a turma e exibe todos os alunos
    print("\nREGISTROS PERCORRENDO AS KEYS:")
    for id_aluno, dados in turma.items():
        exibir_registro_aluno(id_aluno, dados)

# Uso
turma = {
    "aluno1": {"nome": "João", "nota": 7.5},
    "aluno2": {"nome": "Maria", "nota": 9.2},
}
exibir_turma(turma)
```

Output:

```
REGISTROS PERCORRENDO AS KEYS:
ID: aluno1
Nome: João
Nota: 7.5
-----
ID: aluno2
Nome: Maria
Nota: 9.2
-----
```

Dicionários com listas

Nesta situação, em um dicionário os values são listas.

Exemplo, considere uma situação onde cada aluno tem 3 notas e deve ser calculada as médias.

Code:

```
# Dicionário: chave = nome do aluno, valor = lista de notas
alunos = {
    "João": [7.5, 8.0, 9.0],
    "Maria": [9.2, 8.8, 9.5],
    "Pedro": [6.0, 7.0, 7.5],
}

# Acessando todas as notas de Maria
print(f"Notas da Maria: {alunos["Maria"]}")

# Calculando a média do João
media_joao = sum(alunos["João"]) / len(alunos["João"])
print(f"Média do João: {media_joao:.2f}")
```

OUTPUT:

```
Notas da Maria: [9.2, 8.8, 9.5]
Média do João: 8.17
```

Dicionários com listas

Exibindo as informações de forma mais bem formatada:

Code:

```
# Dicionário com listas como valores
alunos = {
    "João": [7.5, 8.0, 9.0],
    "Maria": [9.2, 8.8, 9.5],
    "Pedro": [6.0, 7.0, 7.5]
}
print("=== Lista de Alunos e Notas ===\n")
# Percorrendo o dicionário
for nome, notas in alunos.items():
    # Calcula média
    media = sum(notas) / len(notas)
    # Exibe de forma clara
    print(f"Aluno: {nome}")
    print(f"Notas: {' , '.join(map(str, notas))}")
    print(f"Média: {media:.2f}")
    print("-" * 30)
```

OUTPUT:

```
=== Lista de Alunos e Notas ===

Aluno: João
Notas: 7.5, 8.0, 9.0
Média: 8.17
-----

Aluno: Maria
Notas: 9.2, 8.8, 9.5
Média: 9.17
-----

Aluno: Pedro
Notas: 6.0, 7.0, 7.5
Média: 6.83
-----
```

Dicionários com listas

(em subalgoritmo)

Code:

Aplicando subalgoritmo no código:

```
# Função: Cria o dicionário de alunos
def criar_dicionario() -> None:
    return {
        "João": [7.5, 8.0, 9.0],
        "Maria": [9.2, 8.8, 9.5],
        "Pedro": [6.0, 7.0, 7.5]
    }

# Função: Calcula a média das notas
def calcular_media(notas: list) -> float:
    return sum(notas) / len(notas)

# Procedimento: Exibe informações de um aluno
def exibir_aluno(nome:str , notas:list) -> None:
    media = calcular_media(notas)
    print(f"Aluno: {nome}")
    print(f"Notas: {' , '.join(map(str, notas))}")
    print(f"Média: {media:.2f}")
    print("-" * 30)

# Procedimento: Lista todos os alunos
def listar_alunos(alunos: dict) -> None:
    print("=== Lista de Alunos e Notas ===\n")
    for nome, notas in alunos.items():
        exibir_aluno(nome, notas)

# Uso
turma = criar_dicionario()
listar_alunos(turma)
```

OUTPUT:

```
=== Lista de Alunos e Notas ===

Aluno: João
Notas: 7.5, 8.0, 9.0
Média: 8.17
-----

Aluno: Maria
Notas: 9.2, 8.8, 9.5
Média: 9.17
-----

Aluno: Pedro
Notas: 6.0, 7.0, 7.5
Média: 6.83
-----
```

Lista de dicionários

Podemos também criar uma “Tabela” (lista de dicionário).

No exemplo abaixo são digitados dois registros (dict) e inseridos em uma tabela (list):

Code:

```
# Inicia as estruturas
tabela = [] # lista
pessoa = {} # dicionario

# Preenche e grava dois registros
print("Cadastrando os registros:")
for i in range(2):
    pessoa["nome"] = input("Nome: ")
    pessoa["idade"] = int(input("Idade: "))
    tabela.append(pessoa.copy())
    print()

print("\nExibindo a tabela:")
# Exibe a tabela
for registro in tabela:
    for k, v in registro.items():
        print(f"{k} -> {v}")
    print()
```

Output:

Cadastrando os registros:

Nome: Edson

Idade: 34

Nome: Maria

Idade: 22

Exibindo a tabela:

nome -> Edson

idade -> 34

nome -> Maria

idade -> 22

Lista de dicionários

Code:

```
def preenche_registro(p: dict) -> None:
    # Cria o dicionário e preenche as Keys
    p["nome"] = input("Nome: ")
    p["idade"] = int(input("Idade: "))

def add_registro(t: list, p: dict) -> None:
    # Grava o registro (item na lista)
    t.append(p.copy())

def exibe_tabela(t: list) -> None:
    for registro in t:
        for k, v in registro.items():
            print(f"{k} -> {v}")
        print(30*'-' )

# Uso
tabela = [] # lista
pessoa = {} # dicionario
# Adiciona dois registros na tabela
print(f"Preenchendo 2 registros:\n" +
      f"{len('Preenchendo 2 registros:')*'-'}")
preenche_registro(pessoa)
add_registro(tabela, pessoa)
preenche_registro(pessoa)
add_registro(tabela, pessoa)

# Exibe a tabela
print(f"\nExibindo Registros\n" +
      f"{len('Exibindo Registros')*'-'}")
exibe_tabela(tabela)
```

algoritmo)

Veja o mesmo código com subalgoritmo aplicado:

Output:

Preenchendo 2 registros:

Nome: Edson de Oliveira

Idade: 23

Nome: Maria da Silva

Idade: 33

Exibindo Registros

nome -> Edson de Oliveira

idade -> 23

nome -> Maria da Silva

idade -> 33

BeabaProgramAcao

Prof. Edson de Oliveira



@BeabaProgramAcao



@BeabaDaProgramacaoEds